

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчёт

Лабораторная работа 3

Контейнеризация микросервисов

Выполнил:

Шестак Б.Е.

К3340

Проверил:

Добряков Д.И.

Санкт-Петербург

2026 г.

Задача

Реализовать Dockerfile для каждого микросервиса платформы поиска работы, написать общий docker-compose.yml, настроить сетевое взаимодействие между сервисами с учётом интеграции RabbitMQ.

1. Структура проекта

Проект содержит 5 микросервисов и брокер сообщений, каждый в отдельной директории:

```
lab3/
├── docker-compose.yml
├── auth-service/
│   ├── Dockerfile
│   ├── requirements.txt
│   └── app/
├── vacancy-service/
│   ├── Dockerfile
│   ├── requirements.txt
│   └── app/
├── resume-service/
│   ├── Dockerfile
│   ├── requirements.txt
│   └── app/
├── application-service/
│   ├── Dockerfile
│   ├── requirements.txt
│   └── app/
└── api-gateway/
    ├── Dockerfile
    ├── requirements.txt
    └── main.py
```

2. Dockerfile

Все сервисы используют единый шаблон Dockerfile на базе python:3.11-slim. Образ slim выбран для минимизации размера контейнера. Зависимости устанавливаются отдельным слоем до копирования кода — это позволяет Docker кэшировать слой с зависимостями и ускоряет пересборку при изменении только кода.

Пример Dockerfile для сервисов с БД (auth, vacancy, resume, application):

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN mkdir -p /data
EXPOSE 800X
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "800X"]
```

Директория /data создаётся для хранения SQLite-файлов баз данных. Она монтируется как именованный том в docker-compose.yml для сохранности данных между перезапусками контейнеров.

Таблица 1 — Порты сервисов

Сервис	Порт внутри контейнера	Проброс на хост
api-gateway	8000	0.0.0.0:8000
auth-service	8001	0.0.0.0:8001
vacancy-service	8002	0.0.0.0:8002
resume-service	8003	0.0.0.0:8003

Сервис	Порт внутри контейнера	Проброс на хост
application-service	8004	0.0.0.0:8004
rabbitmq	5672 / 15672	0.0.0.0:5672 / 0.0.0.0:15672

3. Docker Compose

Файл `docker-compose.yml` описывает все 6 сервисов (включая RabbitMQ) и обеспечивает их оркестрацию.

Ключевые решения:

- Общая сеть `job_network` с драйвером `bridge` — все контейнеры общаются по имени сервиса (`auth-service`, `rabbitmq` и т.д.) без необходимости знать IP-адреса
- Именованные тома `auth_data`, `vacancy_data`, `resume_data`, `app_data`, `rabbitmq_data` — данные сохраняются при пересборке контейнеров
- `healthcheck` для `auth-service` и `rabbitmq` — другие сервисы стартуют только после проверки готовности зависимостей
- `depends_on` с `condition: service_healthy` — гарантирует правильный порядок запуска

Порядок запуска сервисов:

```

rabbitmq      → (healthcheck: rabbitmq-diagnostics ping)
auth-service  → (healthcheck: GET /health)
vacancy-service → depends_on: auth-service (healthy) + rabbitmq (healthy)
resume-service → depends_on: auth-service (healthy) + rabbitmq (healthy)
application-service → depends_on: все три выше (healthy)
api-gateway   → depends_on: все четыре сервиса

```

4. Сетевое взаимодействие

Все контейнеры подключены к сети `job_network`. Внутри сети сервисы обращаются друг к другу по имени контейнера:

Таблица 2 — Переменные окружения для сетевого взаимодействия

Сервис	Переменная	Значение
vacancy-service	AUTH_SERVICE_URL	http://auth-service:8001
resume-service	AUTH_SERVICE_URL	http://auth-service:8001
application-service	AUTH_SERVICE_URL	http://auth-service:8001
application-service	VACANCY_SERVICE_URL	http://vacancy-service:8002
application-service	RESUME_SERVICE_URL	http://resume-service:8003
application-service	RABBITMQ_URL	amqp://guest:guest@rabbitmq:5672/
vacancy-service	RABBITMQ_URL	amqp://guest:guest@rabbitmq:5672/
resume-service	RABBITMQ_URL	amqp://guest:guest@rabbitmq:5672/
api-gateway	APP_SERVICE_URL	http://application-service:8004

Внешние порты пробрасываются только для разработки и тестирования. В `production` окружении внутренние сервисы не должны быть доступны снаружи — только `api-gateway` на порту 8000.

5. Запуск и проверка

Запуск всего стека одной командой:

```
docker-compose up --build
```

Проверка состояния контейнеров:

```
docker-compose ps
```

Результат успешного запуска — все 6 контейнеров в статусе Up:

NAME	STATUS
rabbitmq	Up (healthy)
auth-service	Up (healthy)
vacancy-service	Up (healthy)
resume-service	Up (healthy)
application-service	Up
api-gateway	Up

Swagger UI доступен по адресам:

- <http://localhost:8000/docs> — API Gateway (единая точка входа)
- <http://localhost:8001/docs> — auth-service
- <http://localhost:8002/docs> — vacancy-service
- <http://localhost:8003/docs> — resume-service
- <http://localhost:8004/docs> — application-service
- <http://localhost:15672> — RabbitMQ Management UI (guest/guest)

6. Тестирование

Сквозное тестирование проведено через Postman с базовым URL <http://localhost:8000>:

- Регистрация и авторизация пользователей (соискатель и работодатель) — успешно (201/200)
- Создание компании и вакансии работодателем — успешно (201)
- Создание резюме соискателем — успешно (201)
- Подача заявки на вакансию — успешно (201), межсервисные проверки отработали
- В RabbitMQ Management UI обе очереди отображают Total = 0 — события обработаны

— Смена статуса заявки работодателем — успешно, событие `application.status_changed` доставлено

Вывод

В ходе выполнения лабораторной работы реализована полная контейнеризация микросервисной архитектуры платформы поиска работы. Написаны Dockerfile для каждого из 5 сервисов на базе python:3.11-slim. Создан docker-compose.yml с описанием 6 сервисов (включая RabbitMQ), общей сетью job_network, именованными томами для хранения данных и healthcheck для контроля порядка запуска. Все сервисы успешно запускаются одной командой docker-compose up --build и корректно взаимодействуют между собой через Docker-сеть.